

TeamViewer Full-Stack Engineer Interview Guide

Comprehensive Technical Reference: C#/.NET Backend, Angular/TypeScript Frontend & Systems Architecture

1. C# & .NET Core Backend Architecture

Q: Explain inheritance vs encapsulation. Compare their differences, similarities, and how they impact code maintainability.

Answer:

- **Inheritance:** A mechanism where a child class acquires properties and behavior from a parent class, facilitating code reuse through an 'is-a' relationship.
- **Encapsulation:** Hiding the internal state/representation of an object and restricting direct access via access modifiers, promoting a structural contract via an 'has-a' or behavioral interaction.
- **Impact on Maintainability:** Overuse of inheritance creates tight coupling (the fragile base class problem). Encapsulation isolates changes within specific modules, lowering side effects when changing internal system rules.

Q: How does the async/await state machine handle threading behind the scenes in .NET?

Answer: When a method uses async/await, the compiler builds an internal state machine structure that implements `IAsyncStateMachine`. Instead of blocking the executing thread during an IO-bound operation, the thread is released back to the `ThreadPool`. When the awaited task completes, execution resumes on a thread determined by the captured `SynchronizationContext` (unless `ConfigureAwait(false)` is explicitly declared). Using `Task.Run()` offloads heavy CPU-bound tasks onto a completely separate `ThreadPool` worker thread asynchronously, avoiding blockage of critical primary or UI thread paths.

Q: How do you manage unmanaged resources securely via the `IDisposable` pattern?

Answer: Implement the explicit standard disposal pattern to ensure resources clean up under deterministic execution windows or safely via garbage collection fallbacks:

```
public class ResourceHandler : IDisposable
{
    private bool _disposed = false;
    private SafeHandle _unmanagedResource; // Example unmanaged reference

    public void Dispose() {
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    protected virtual void Dispose(bool disposing) {
        if (_disposed) return;
        if (disposing) {
            // Free managed resources here
        }
        // Free unmanaged resources here
        _disposed = true;
    }
}
```

```
    }  
  
    ~ResourceHandler() => Dispose(false);  
}
```

Q: How do you handle optimistic vs pessimistic concurrency in Entity Framework Core?

Answer:

- **Optimistic Concurrency:** Assumes conflicts are rare. EF Core tracks properties configured with a [Timestamp] or IsRowVersion() token. If data updates concurrently, a DbUpdateConcurrencyException triggers, forcing resolution code loops.
- **Pessimistic Concurrency:** Assumes conflicts are highly frequent. EF Core locks databases explicitly at data extraction (e.g., executing raw SQL statements containing transaction locks like WITH (XLOCK) or using explicit isolation scopes).

2. Angular & TypeScript Frontend

Q: Explain the execution order of `ngOnInit`, `ngOnChanges`, and `ngAfterViewInit`.

Answer: The chronological sequence executes as `ngOnChanges` -> `ngOnInit` -> `ngAfterViewInit`. The `ngOnChanges` hook fires first because Angular sets the component's data-bound `@Input` properties before running initial instantiation routines. **Gotcha:** If an input property receives an internal property mutability edit within a parent object directly without changing the object reference footprint, `ngOnChanges` will **not** fire because Angular verifies references using strict identity equality (`===`).

Q: How do you manage memory leaks when handling long-lived RxJS Observables?

Answer: Use the `takeUntil` operator tied directly to a lifecycle notifier subject to automate unsubscribe handling cleanly upon component destruction:

```
@Component({ selector: 'app-user' })
export class UserComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject();

  ngOnInit() {
    this.userService.getData().pipe(
      takeUntil(this.destroy$)
    ).subscribe(data => console.log(data));
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

Q: How does `ChangeDetectionStrategy.OnPush` improve performance?

Answer: `OnPush` deactivates default dirty check traversals for sub-component UI sub-trees. Angular only triggers change detection for the template when an `@Input` reference alters completely, an internal event fires from its template, or an Observable bound via the `async` pipe emits. To execute updates outside the main zone context, inject `ChangeDetectorRef` explicitly and call `detectChanges()` manually.

3. Full-Stack Integration & Web Basics

Q: What happens when you enter a website name in a browser?

Answer: The browser requests translation via local DNS or global nameservers. Once resolved, a TCP handshake establishes a connection. A TLS handshake negotiates encryption keys. The HTTP request hits a load balancer/reverse proxy, which forwards it to the hosting server. The server processes the request (e.g., executing middleware, querying database layers) and returns the asset package to render.

Q: Explain dynamic path object dictionary flattening.

Answer: Below is an optimized TypeScript utility to flatten complex structures into dot-delimited path structures efficiently:

```
function flattenObject(obj: any, prefix = '', res: any = {}): any {
  for (const key in obj) {
    if (obj.hasOwnProperty(key)) {
      const propName = prefix ? `${prefix}.${key}` : key;
      if (typeof obj[key] === 'object' && obj[key] !== null && !Array.isArray(obj[key])) {
```

```
        flattenObject(obj[key], propName, res);
    } else {
        res[propName] = obj[key];
    }
}
}
return res;
}
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.